Linear Regression

Regression

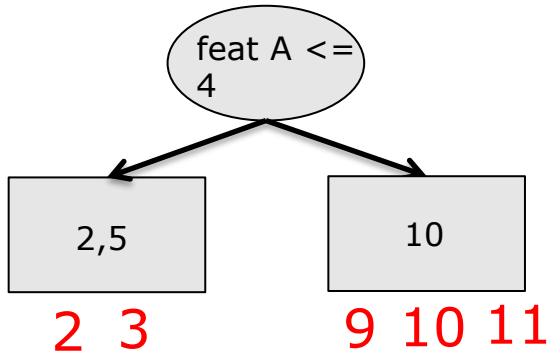
- Regression
  - Predicting value instead of class/category.
  - Labels Y=R (real values).

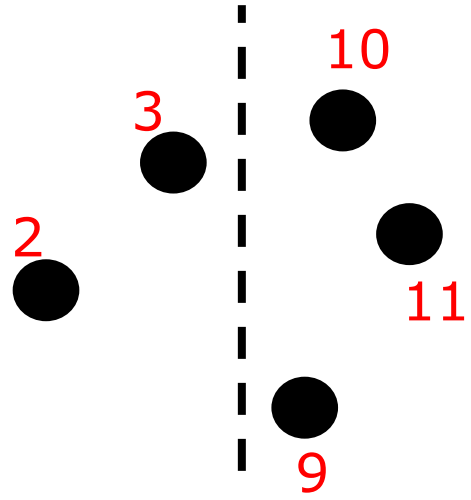


- Predict house prices.

Small Regression Trees

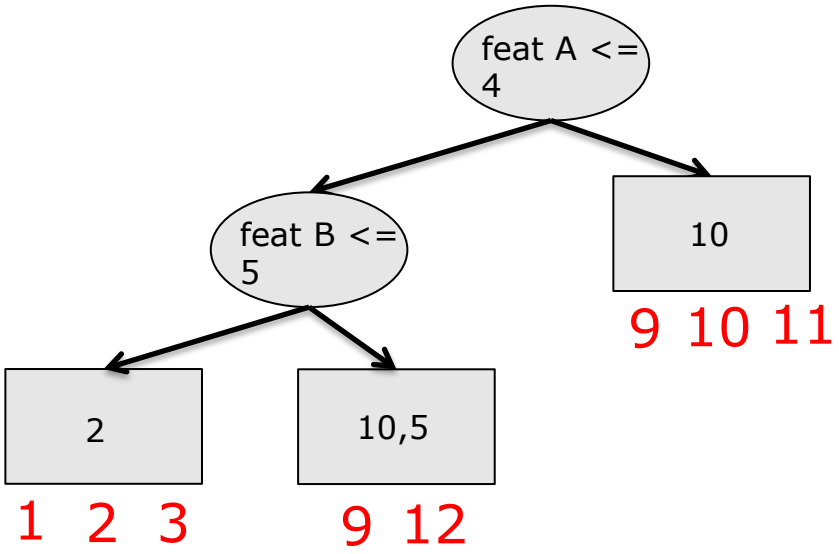
- One Approach:
  - Use regression trees.

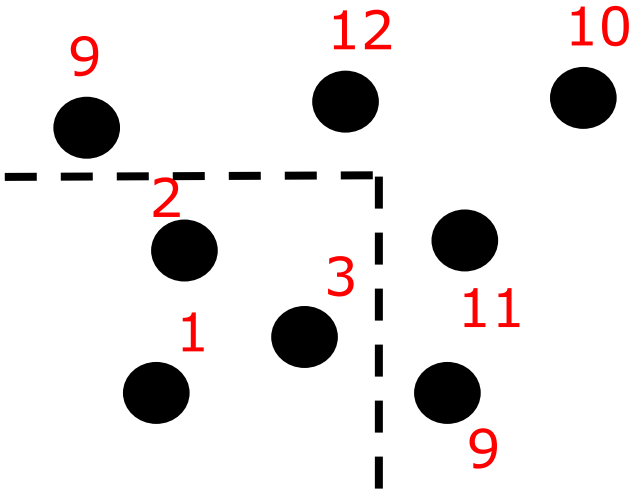




Small Regression Trees

- Regression Trees:
  - Good for some types of data

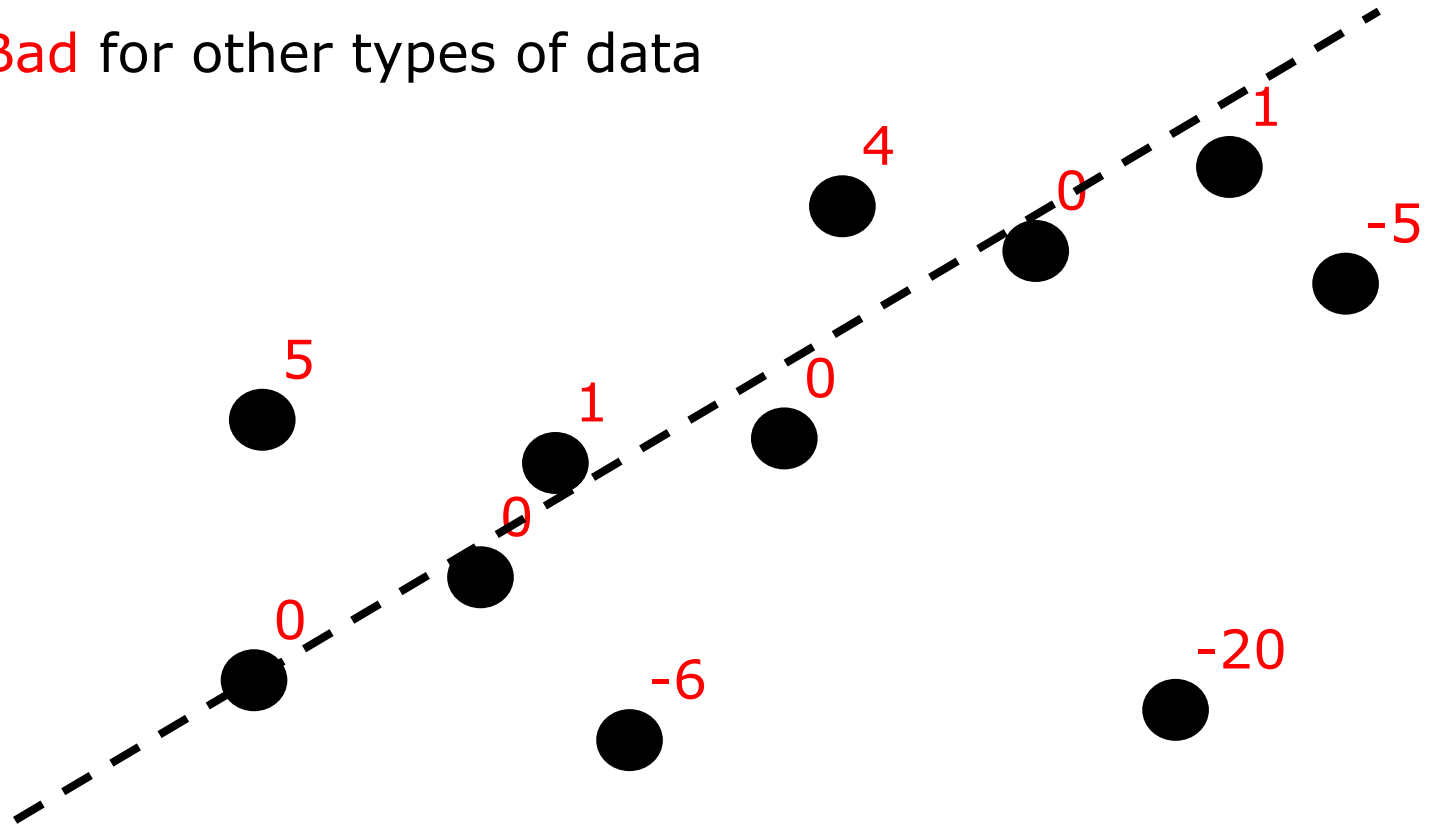




Small Regression Trees

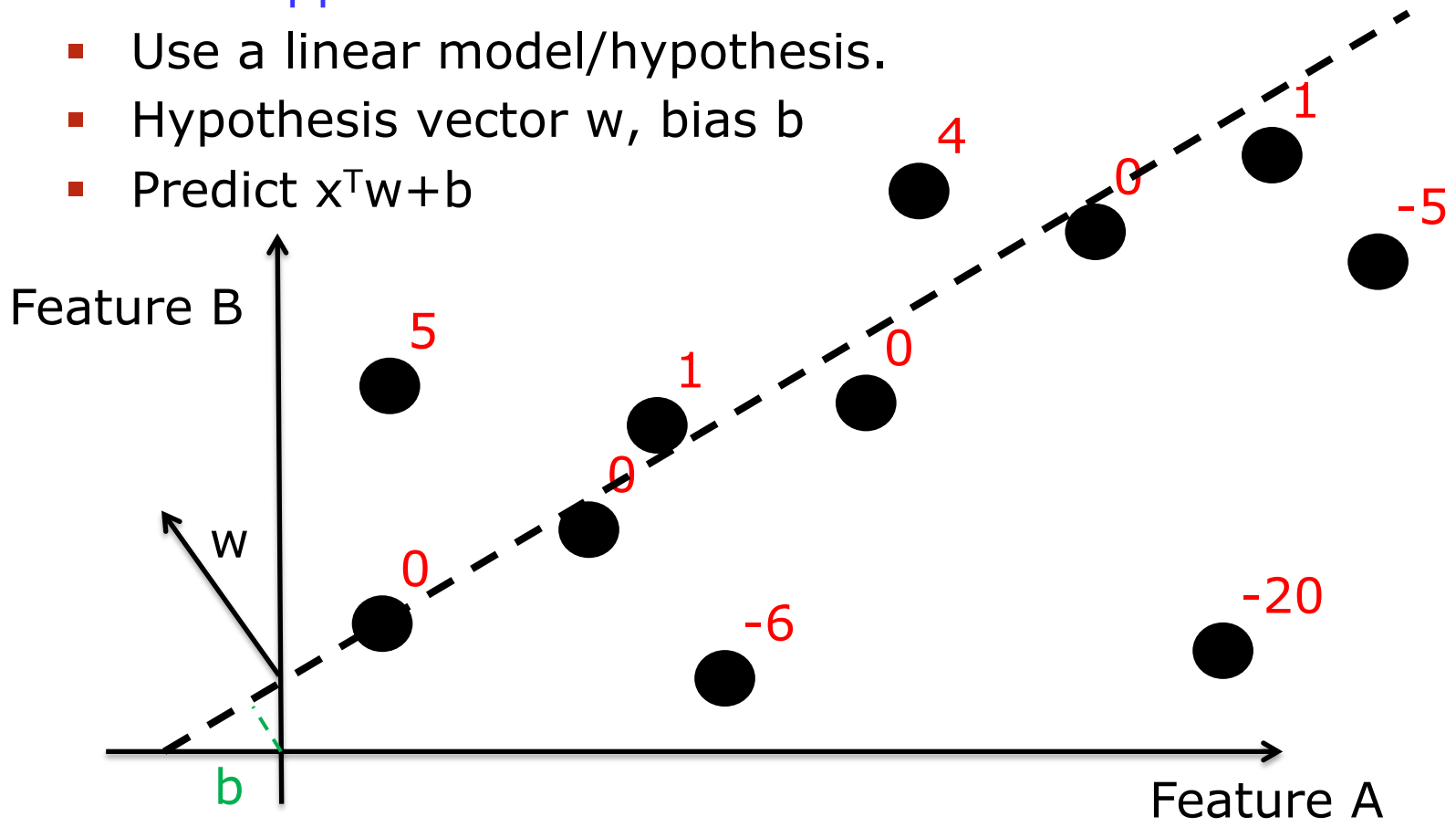
Maybe distance to this line?

- Regression Trees:
  - Bad for other types of data



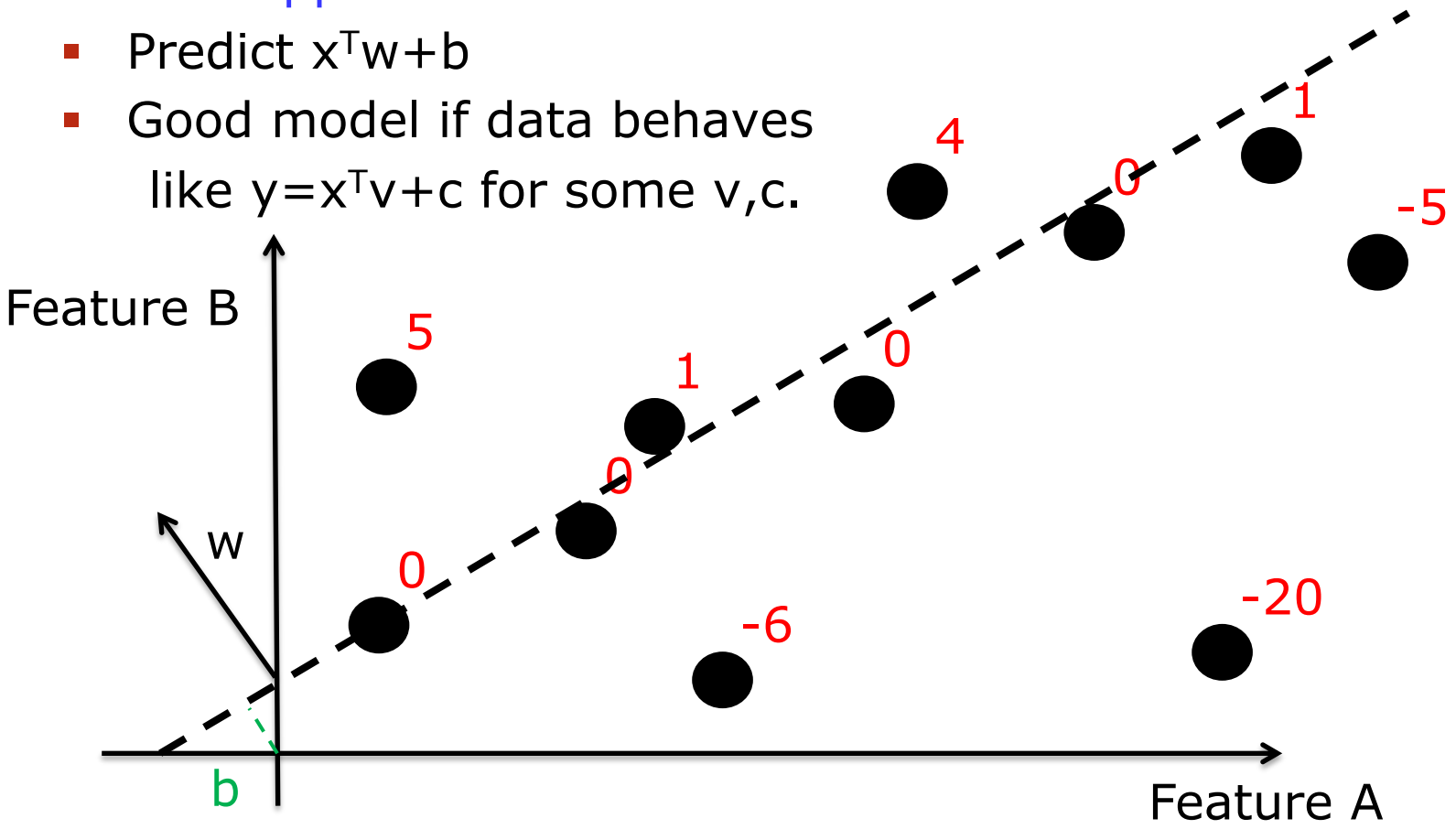
Linear Regression

- Another Approach:
  - Use a linear model/hypothesis.
  - Hypothesis vector w, bias b
  - Predict $x^T w+b$



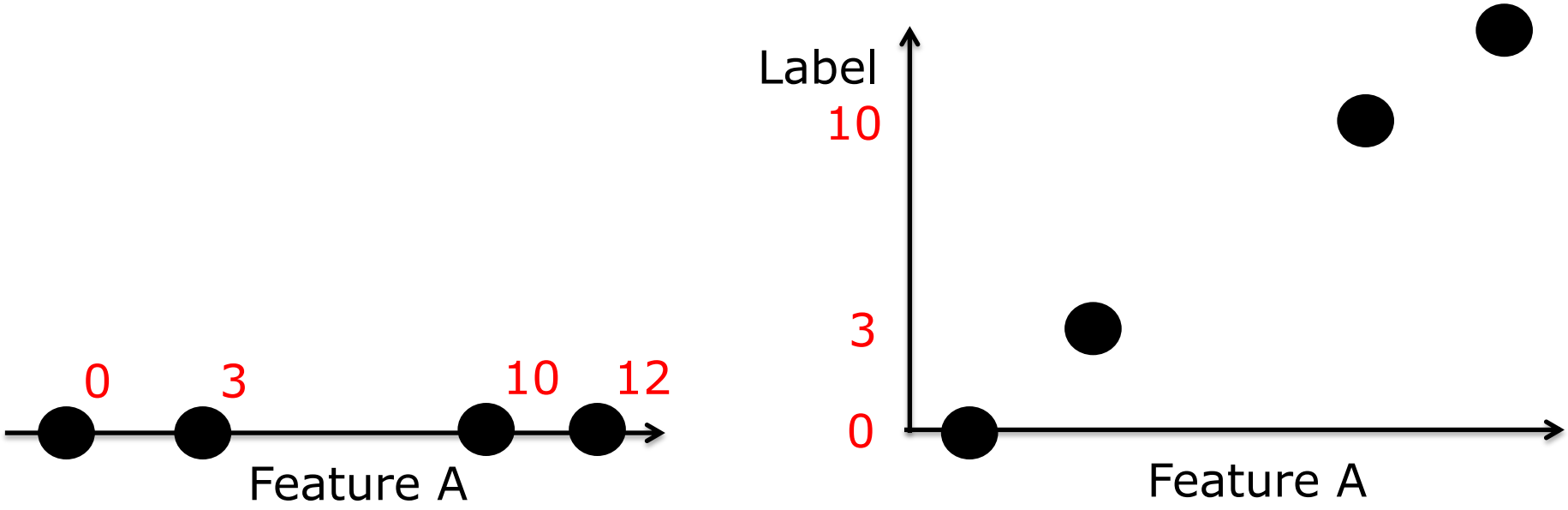
Linear Regression

- Another Approach:
  - Predict $x^T w+b$
  - Good model if data behaves like $y=x^T v+c$ for some v,c.



Easier Visualization

- For linear regression, let us visualize differently:
  - Plot with label as last coordinate.
  - One-dimensinal example (one feature).



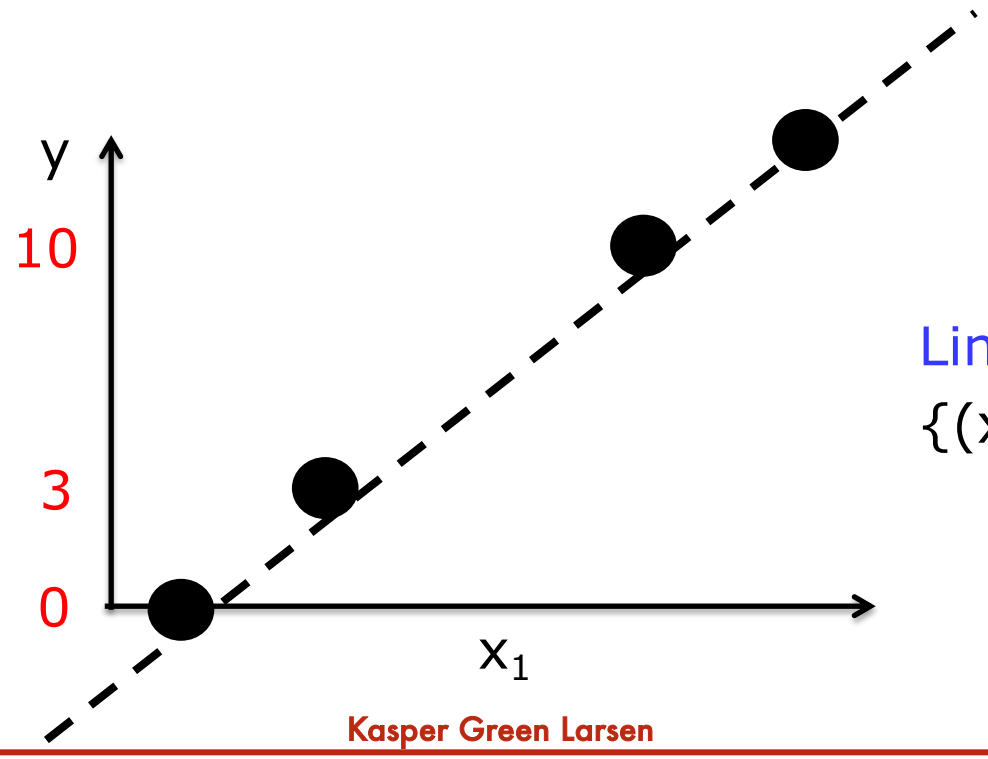
Easier Visualization

- Linear regression. If d features, plot in d+1 dims.
  - Predict: $x^T w+b$

Example:
w=(1), b=-2

Predict:
$1x_1-2$

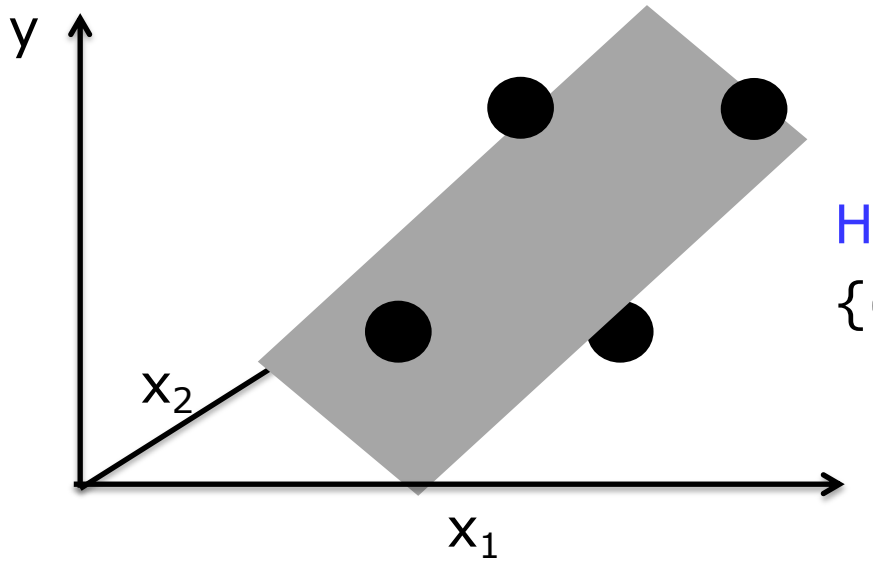
A line!



Line:
$\{(x_1,y) \mid 1x_1-2=y\}$

Easier Visualization

- Linear regression. If d features, plot in d+1 dims.
  - Predict: $x^T w+b$
  - Hyperplane when d>1.
    - Given all d features, predict as (d+1)'st coordinate of point on hyperplane.

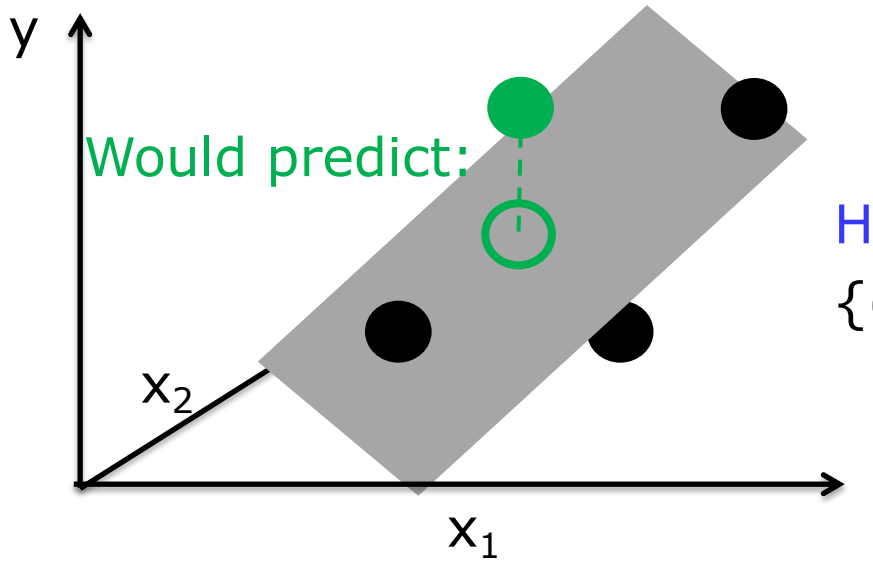


Hyperplane:

$\{(x_1,\ldots,x_d,y) \mid x^T w+b=y\}$

Easier Visualization

- Linear regression. If d features, plot in d+1 dims.
  - Predict: $x^T w+b$
  - Hyperplane when d>1.
    - Given all d features, predict as (d+1)'st coordinate of point on hyperplane.

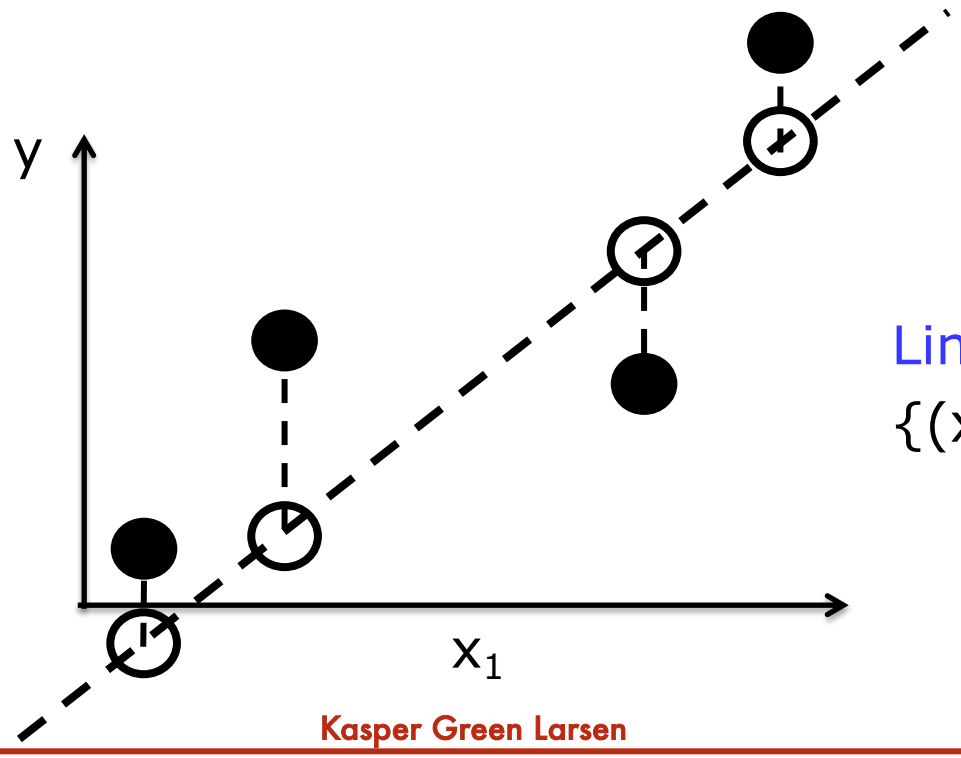


Hyperplane:

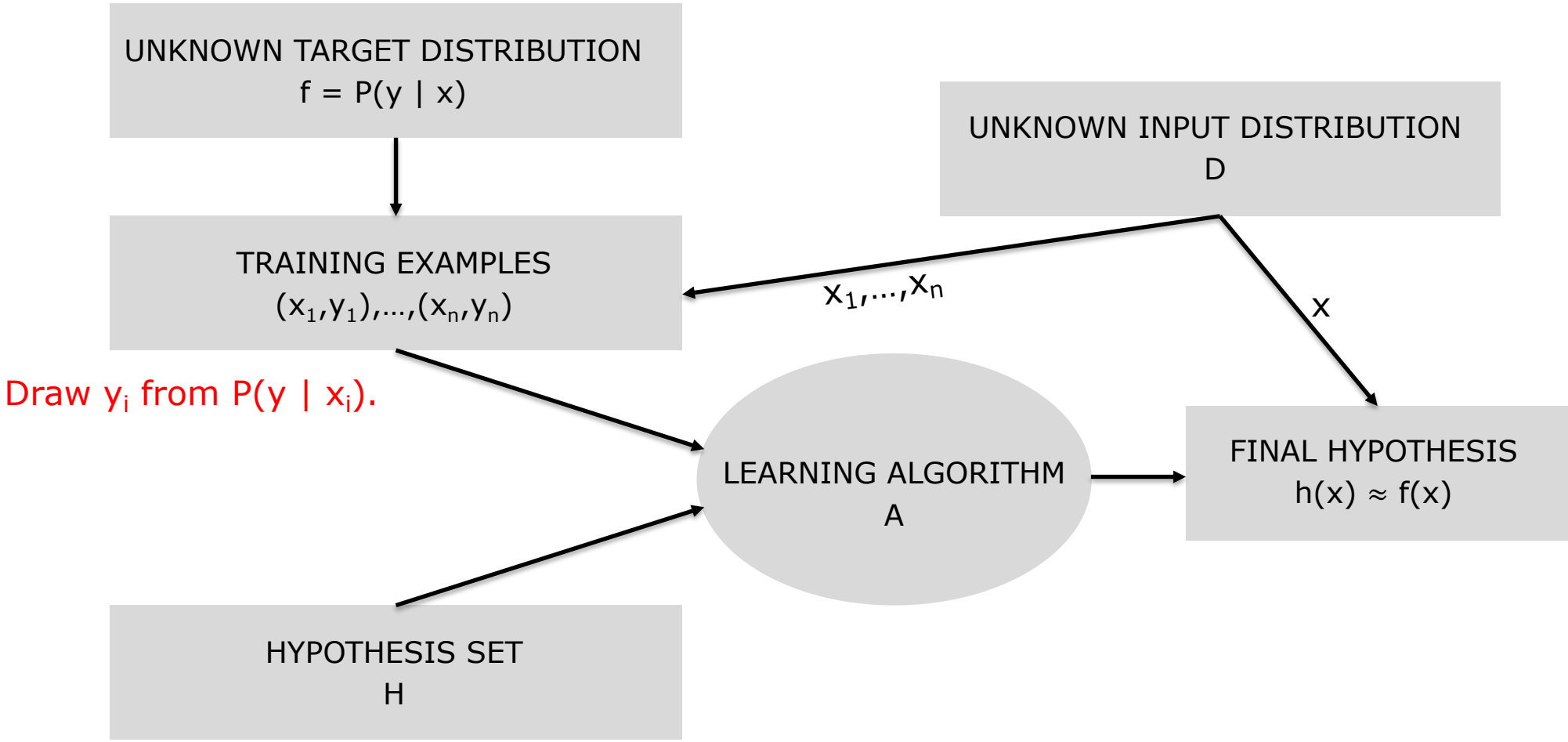
$\{(x_1,\ldots,x_d,y) \mid x^T w+b=y\}$

Easier Visualization

- Linear regression. If d features, plot in d+1 dims.
  - Predict: $x^T w+b$



Supervised Learning



Loss Functions

- Least Squares Loss Function
  - In regression, often use least squares loss.
  - $L(y',y) = (y'-y)^2$



- $L(5.300.000, 5.150.000) = (150.000)^2$

In-Sample Error

- Data Generation:
  - Input domain X, output domain Y
  - Unknown distribution D over X
  - Unknown target distribution P(y | x)
  - Receive n samples $(x_1,y_1),(x_2,y_2),\ldots,(x_n,y_n)$.

x_i drawn from D

y_i drawn from P(y | x_i)

- General loss function L:

$$E_{in}(h) = \frac{1}{n}\sum_{i=1}^{n} L(h(x_i), y_i)$$

- Least Squares:

$$E_{in}(h) = \frac{1}{n}\sum_{i=1}^{n} (h(x_i) - y_i)^2$$

Out-of-Sample Error

- Data Generation:
  - Input domain X, output domain Y
  - Unknown distribution D over X
  - Unknown target distribution P(y | x)

- General loss function L:

$$E_{out}(h) = \mathbb{E}_{D,y}[L(h(x),y)]$$

- Least Squares:

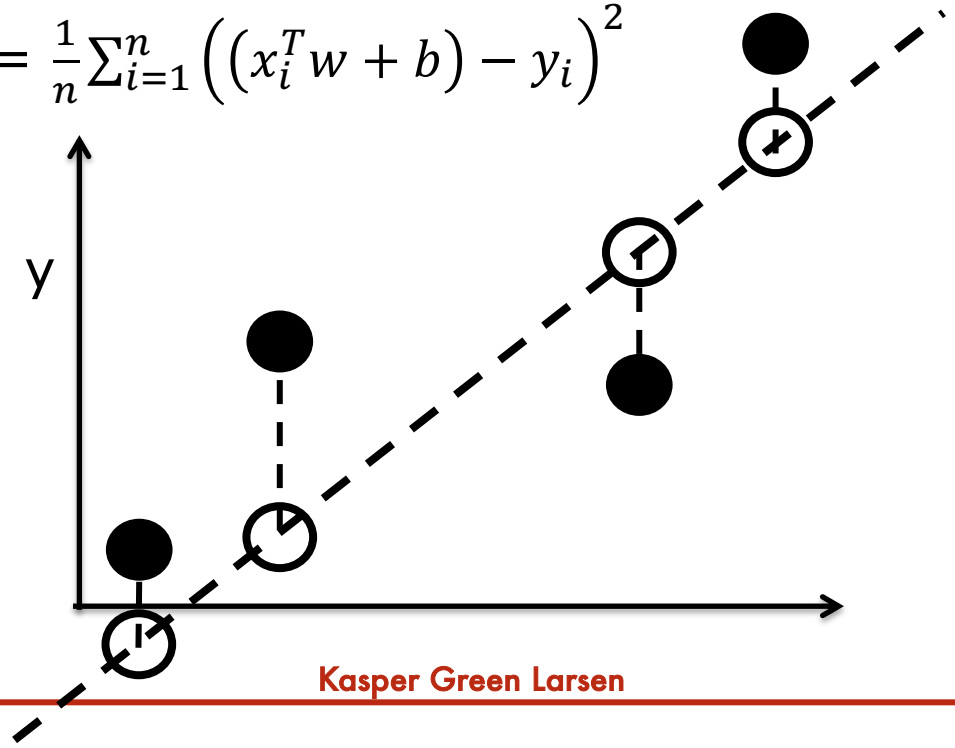
$$E_{out}(h) = \mathbb{E}_{D,y}[(h(x)-y)^2]$$

Linear Regression

- Predict: $x^T w+b$
- Want to minimize:
  - $E_{out}(w,b) = E_{D,y}[((x^T w + b) - y)^2]$
- Instead minimize:
  - $E_{in}(w,b) = \frac{1}{n}\sum_{i=1}^{n}\left(\left(x_i^T w + b\right) - y_i\right)^2$

True label

Prediction



The Trick

- Regression:
  - Features: $x=(x_1,\ldots,x_d)$
  - Labels: R
- Linear Model:
  - Parameters of hypothesis w: $(w_1, w_2,\ldots, w_d, b)$
  - Predict: $w(x) = w_1x_1 + w_2x_2 + \ldots + w_dx_d + b = x^Tw+b$

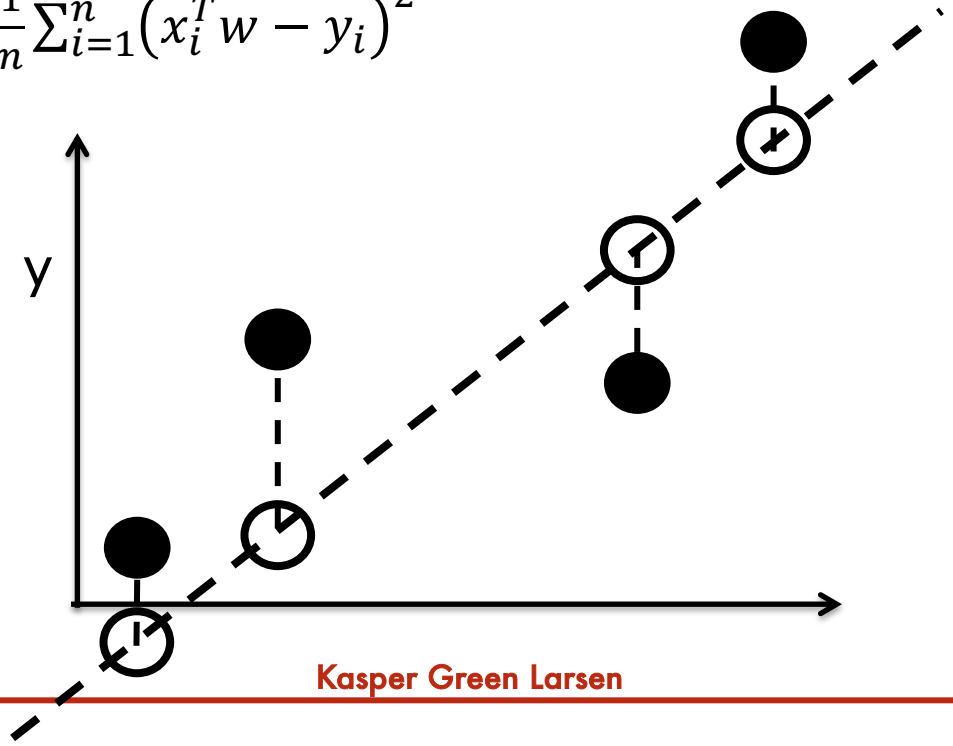
- Trick:
  - Annoying to have special bias parameter b.
  - Make data (d+1)-dimensional and set first coordinate of all data points equal to 1.
  - Then parameters $(w_0, w_1, w_2,\ldots, w_d)$.
  - Predict $w(x) = w_01 + w_1x_1 + w_2x_2 + \ldots + w_dx_d = x^Tw$.

Linear Regression

- Predict: $x^T w$
- Want to minimize:
  - $E_{out}(w) = E_{D,y}[(x^T w - y)^2]$
- Instead minimize:
  - $E_{in}(w) = \frac{1}{n}\sum_{i=1}^{n}\left(x_i^T w - y_i\right)^2$

True label

Prediction

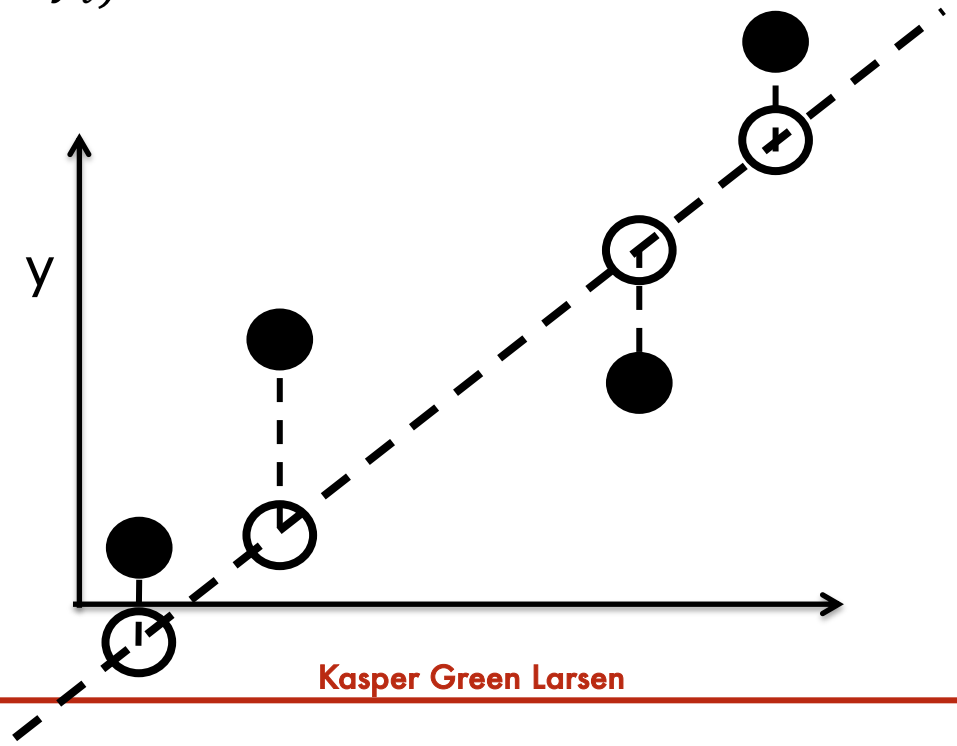


Linear Regression

- Goal: Minimize
  - $E_{in}(w) = \frac{1}{n}\sum_{i=1}^{n}\left(x_i^T w - y_i\right)^2$
- Same as minimizing:
  - $\sum_{i=1}^{n}\left(x_i^T w - y_i\right)^2$

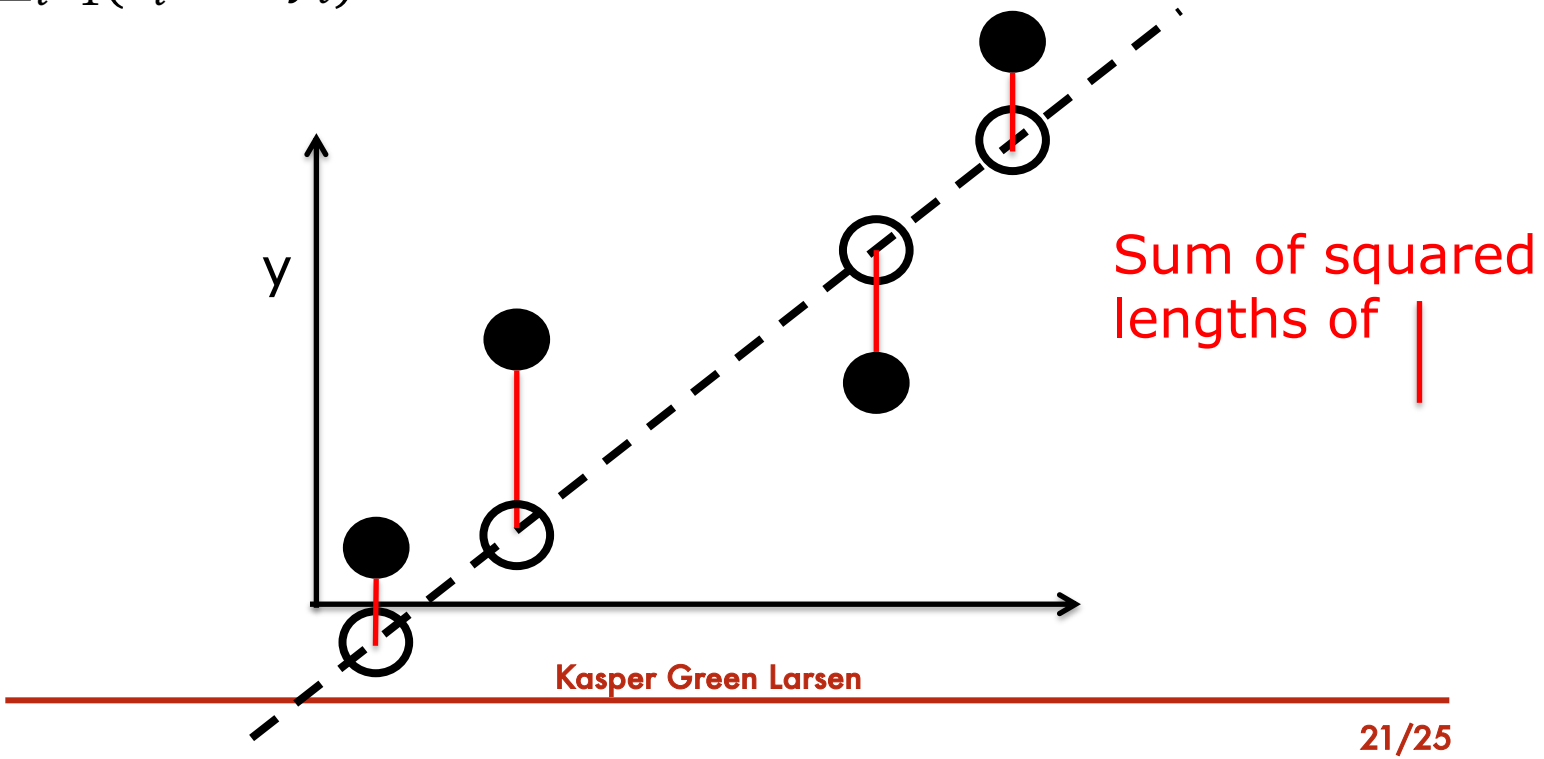
True label

Prediction



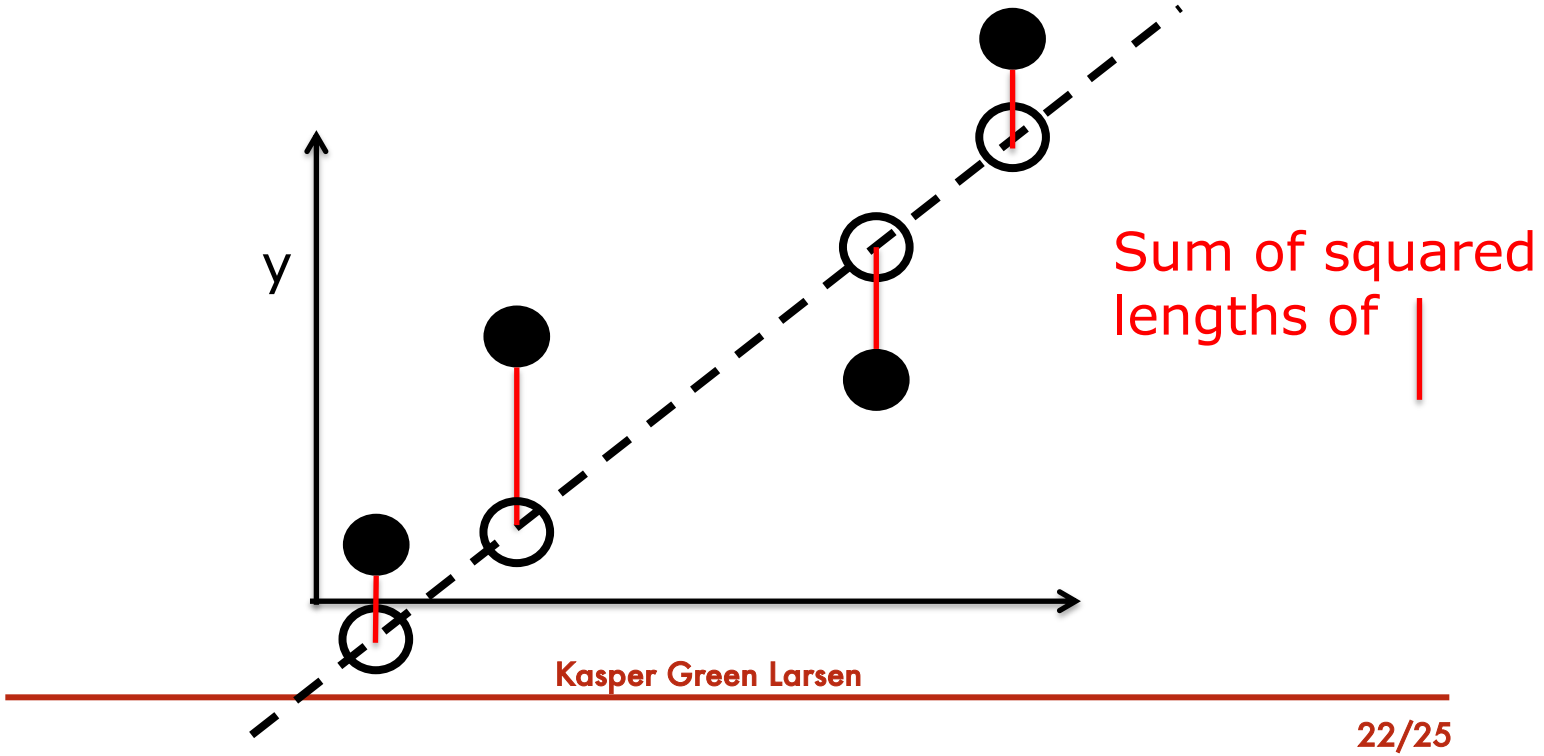
Linear Regression

- Goal: Minimize
  - $E_{in}(w) = \frac{1}{n}\sum_{i=1}^{n}\left(x_i^T w - y_i\right)^2$
- Same as minimizing:
  - $\sum_{i=1}^{n}\left(x_i^T w - y_i\right)^2$



Linear Regression

- Goal: Minimize
  - $\sum_{i=1}^{n}(x_i^T w - y_i)^2$
- Rewriting:
  - Let us write in matrix-vector notation.

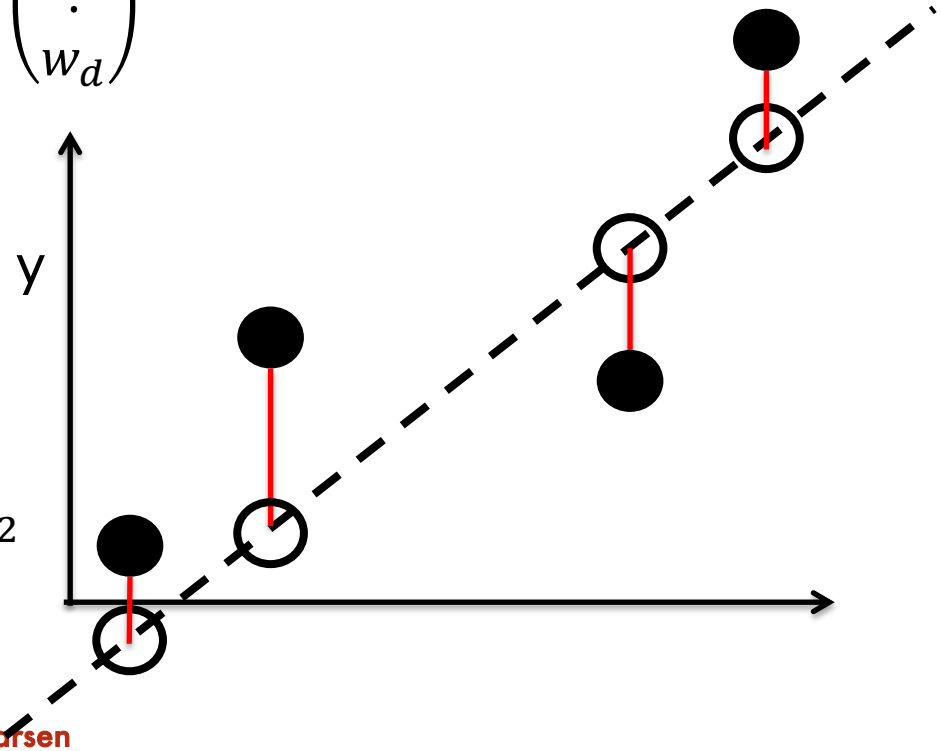


The Classifier Interface

- A Classifier as a Class (Object Oriented Programming).
- Two main methods:
- Fit(X,y): X is a matrix with training data as rows.
  - On data $(x_1,y_1),\ldots,(x_n,y_n)$ then X has n rows with i'th row equal to x_i. So n x d matrix when d features.
  - The vector y has i'th coordinate y_i.
  - Method searches for g with $g(x_i)=y_i$ for most i. Stores g.
- Predict(x): x is a vector of d features.
  - Evaluate and return g(x).

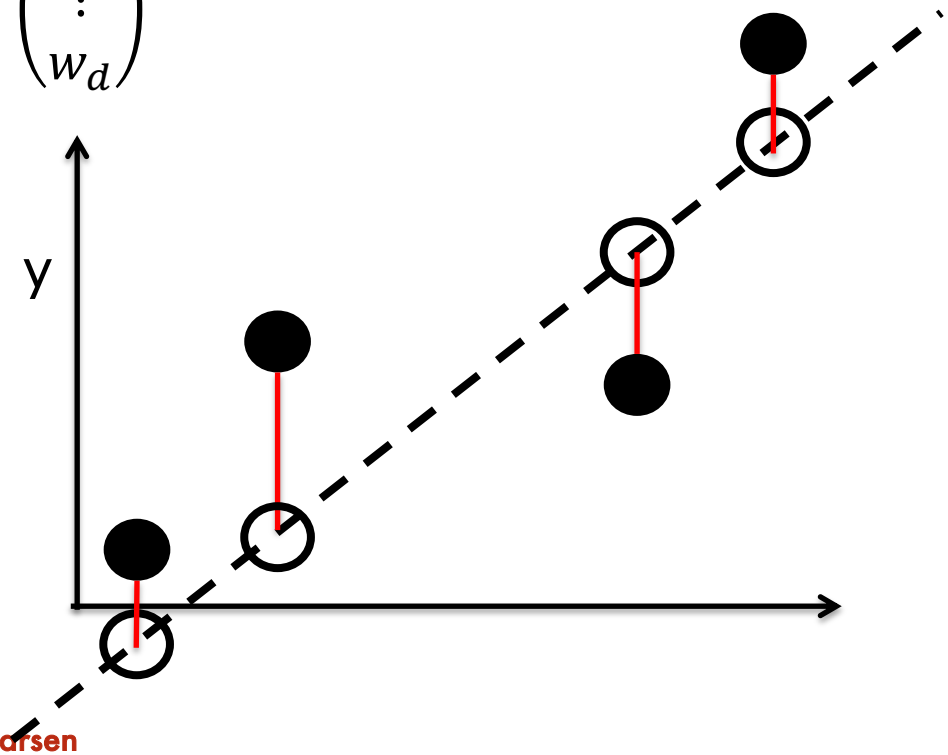
Linear Regression

- Goal: Minimize
  - $\sum_{i=1}^{n}(x_i^T w - y_i)^2$
- $X = \begin{pmatrix} x_{1,1} & \cdots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{n,1} & \cdots & x_{n,d} \end{pmatrix}, y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}, w = \begin{pmatrix} w_1 \\ \vdots \\ w_d \end{pmatrix}$
- $Xw - y = \begin{pmatrix} x_1^T w - y_1 \\ \vdots \\ x_n^T w - y_n \end{pmatrix}$
- $(Xw - y)^T (Xw - y) = \sum_{i=1}^{n}(x_i^T w - y_i)^2$



Linear Regression

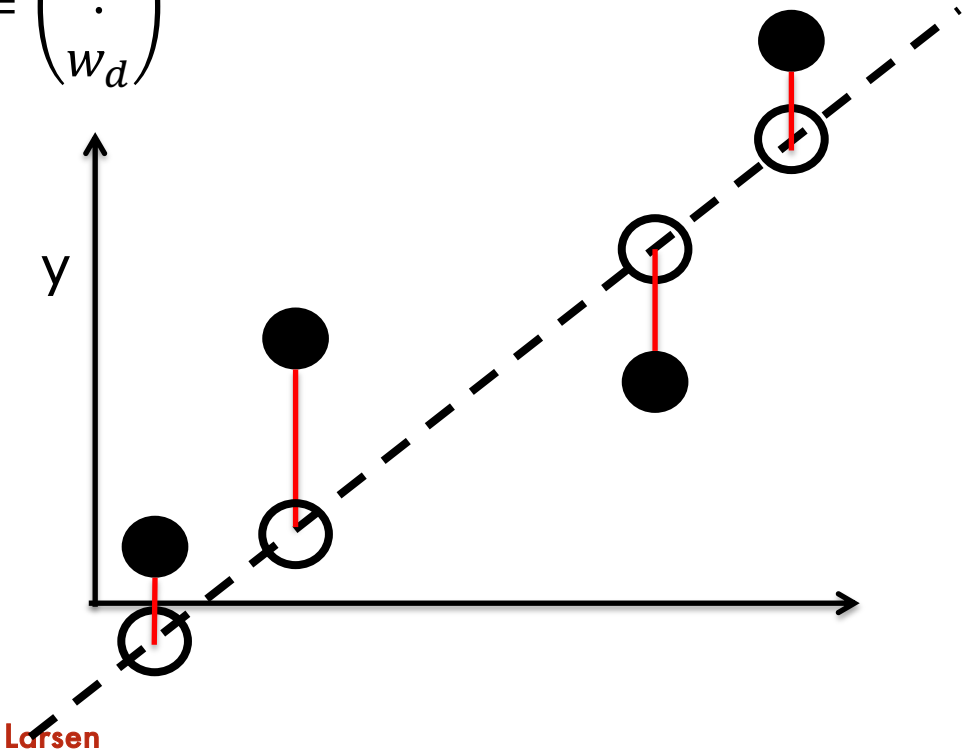
- Goal: Minimize
  - $(Xw - y)^T(Xw - y) = \sum_{i=1}^{n}(x_i^T w - y_i)^2$
- $X = \begin{pmatrix} x_{1,1} & \cdots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{n,1} & \cdots & x_{n,d} \end{pmatrix}, y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}, w = \begin{pmatrix} w_1 \\ \vdots \\ w_d \end{pmatrix}$



Linear Regression

Linear Regression

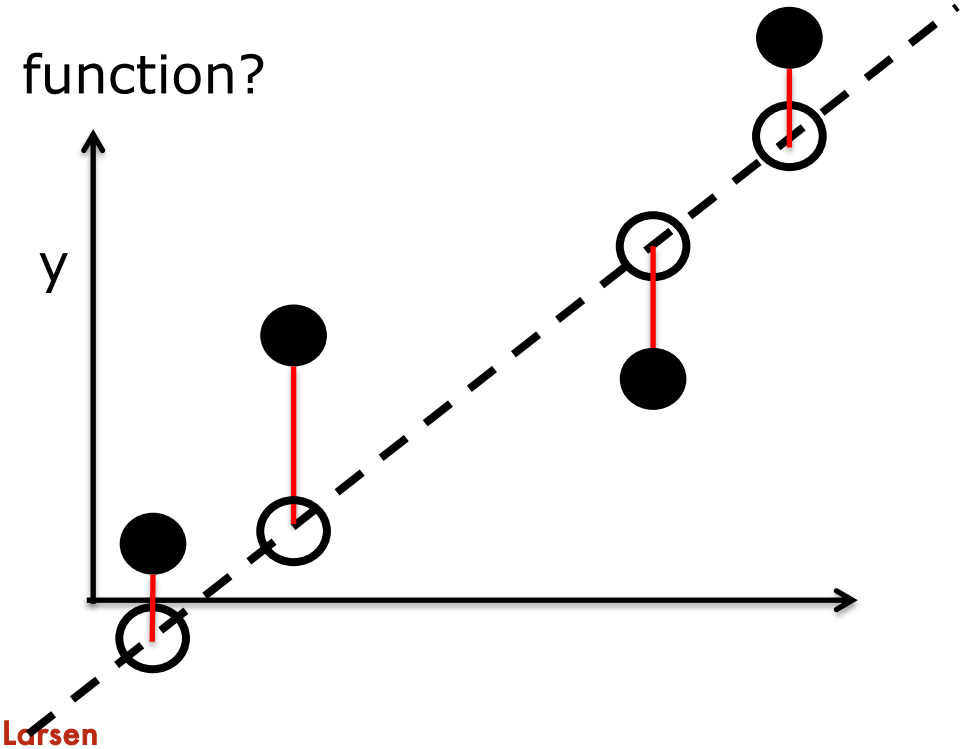
- Goal: Minimize
  - $l(w) = (Xw - y)^T(Xw - y) = \sum_{i=1}^{n}(x_i^T w - y_i)^2$
- $X = \begin{pmatrix} x_{1,1} & \cdots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{n,1} & \cdots & x_{n,d} \end{pmatrix}, y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}, w = \begin{pmatrix} w_1 \\ \vdots \\ w_d \end{pmatrix}$



Linear Regression

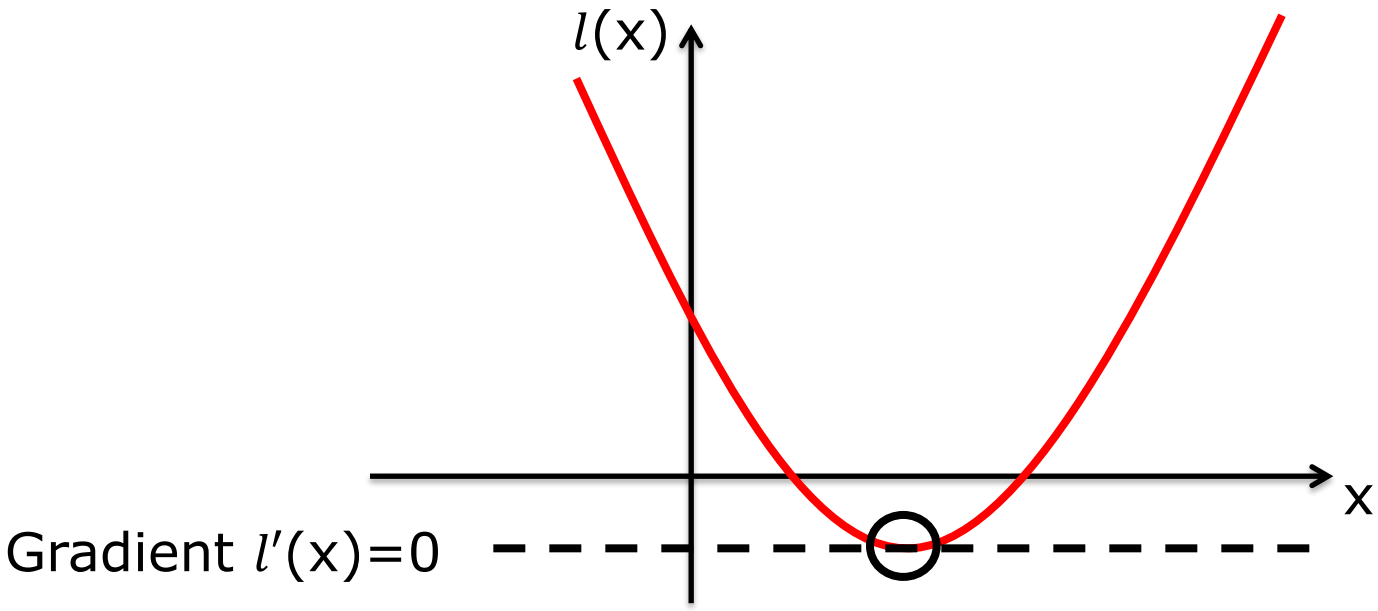
- Goal: Minimize
  - $l(w) = (Xw - y)^T(Xw - y) = \sum_{i=1}^{n}\left(x_i^T w - y_i\right)^2$
  - l is a function from R^d to R.

  - How can we minimize such a function?



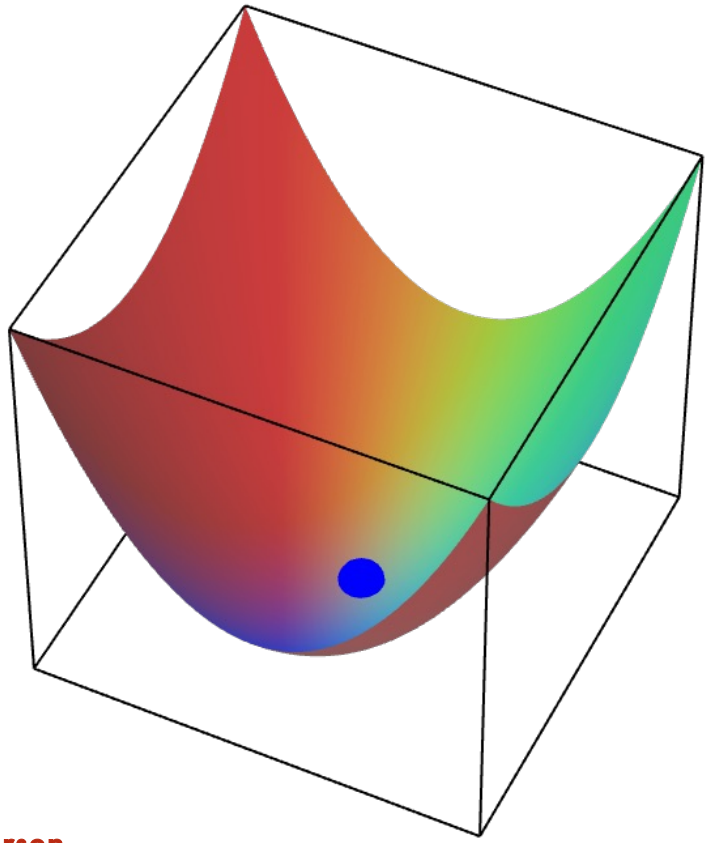
Local Minima

- Functions of one variable:
  - If l is a function of one variable x, i.e. from R to R.
  - Assume l differentiable.



Local Minima

- Functions of multiple variables:
  - If l is a function of variables $x_1,\ldots x_d$, i.e. from R^d to R.
  - Assume l differentiable.



- Gradient (vector):
  - $\nabla_x\big(l(x)\big) = \begin{pmatrix} \frac{\partial}{\partial x_1} l(x) \\ \vdots \\ \frac{\partial}{\partial x_d} l(x) \end{pmatrix}$

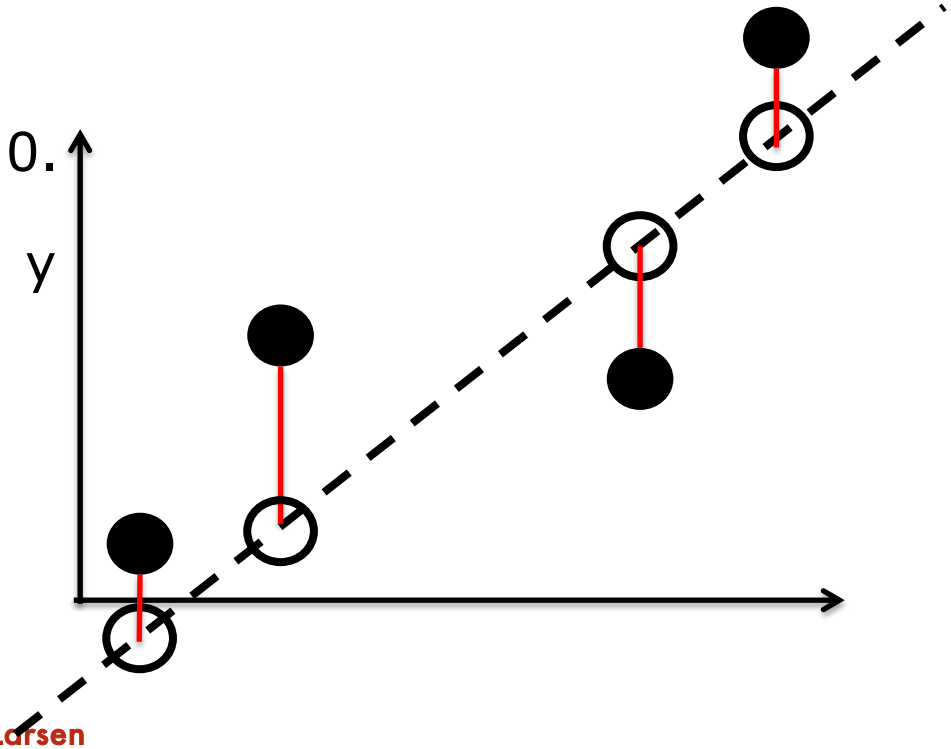
- Local minima must have:
  - $\nabla_x\big(l(x)\big) = 0$

Gradient Vector

- Example:
  - $f(x_1,x_2) = 2x_2^2+x_1$
- Gradient (vector):
  - $\nabla_x(f(x_1,x_2)) = \begin{pmatrix} \frac{\partial}{\partial x_1} 2x_2^2 + x_1 \\ \frac{\partial}{\partial x_2} 2x_2^2 + x_1 \end{pmatrix} = \begin{pmatrix} 1 \\ 4x_2 \end{pmatrix}$

Linear Regression

- Goal: Minimize
  - $l(w) = (Xw - y)^T(Xw - y) = \sum_{i=1}^{n}(x_i^T w - y_i)^2$
  - l is a function from R^d to R.
- Strategy:
  - Compute gradient and set to 0.
  - $\nabla_w l(w) = 0$.
- Will study next time:
  - $l(w)$ is convex.
  - Local minimum is global.
  - No local maximum or saddle points.



Linear Regression

- Goal: Minimize
  - $l(w) = (Xw - y)^T(Xw - y) = \sum_{i=1}^{n}\left(x_i^T w - y_i\right)^2$
  - l is a function from $\mathbb{R}^d$ to $\mathbb{R}$.

- Strategy:
  - $\nabla_w l(w) = \begin{pmatrix} \frac{\partial}{\partial w_1} l(w) \\ \vdots \\ \frac{\partial}{\partial w_d} l(w) \end{pmatrix} = 0.$

- Let us compute j'th entry of $\nabla_w l(w)$.

Linear Regression

- Goal: Minimize
  - $l(w) = (Xw - y)^T(Xw - y) = \sum_{i=1}^{n}(x_i^T w - y_i)^2$
- j'th entry of gradient:
  - $\frac{\partial}{\partial w_j} l(w) = \frac{\partial}{\partial w_j} \sum_{i=1}^{n}(x_i^T w - y_i)^2$
  - $= \sum_{i=1}^{n} \frac{\partial}{\partial w_j} (x_i^T w - y_i)^2$
- Using Chain rule for h(x)=f(g(x)) have h'(x)=f'(g(x))g'(x)
  - $= \sum_{i=1}^{n} 2\,(x_i^T w - y_i)\, \frac{\partial}{\partial w_j}(x_i^T w - y_i)$
  - $= \sum_{i=1}^{n} 2\,(x_i^T w - y_i)\, x_{i,j}$

Linear Regression

- Goal: Minimize
  - $l(w) = (Xw - y)^T(Xw - y) = \sum_{i=1}^{n}\left(x_i^T w - y_i\right)^2$
- j'th entry of gradient:
  - $\frac{\partial}{\partial w_j} l(w) = \sum_{i=1}^{n} 2\,(x_i^T w - y_i)\; x_{i,j}$
- So:
  - $\nabla_w l(w) = \begin{pmatrix} \sum_{i=1}^{n} 2\,(x_i^T w - y_i)\; x_{i,1} \\ \vdots \\ \sum_{i=1}^{n} 2\,(x_i^T w - y_i)\; x_{i,d} \end{pmatrix}$

Linear Regression

- Goal: Minimize
  - $l(w) = (Xw - y)^T(Xw - y) = \sum_{i=1}^{n}\left(x_i^T w - y_i\right)^2$
- j'th entry of gradient:
  - $\frac{\partial}{\partial w_j} l(w) = \sum_{i=1}^{n} 2\,(x_i^T w - y_i)\; x_{i,j}$
- So:
  - $\nabla_w l(w) = \begin{pmatrix} \sum_{i=1}^{n} 2\,(x_i^T w - y_i)\; x_{i,1} \\ \vdots \\ \sum_{i=1}^{n} 2\,(x_i^T w - y_i)\; x_{i,d} \end{pmatrix} = 2\left(\begin{pmatrix} \sum_{i=1}^{n} x_i^T w x_{i,1} \\ \vdots \\ \sum_{i=1}^{n} x_i^T w x_{i,d} \end{pmatrix} - \begin{pmatrix} \sum_{i=1}^{n} y_i x_{i,1} \\ \vdots \\ \sum_{i=1}^{n} y_i x_{i,d} \end{pmatrix}\right).$

Linear Regression

- Goal: Minimize
  - $l(w) = (Xw - y)^T(Xw - y) = \sum_{i=1}^{n}\left(x_i^T w - y_i\right)^2$
- j'th entry of gradient:
  - $\frac{\partial}{\partial w_j} l(w) = \sum_{i=1}^{n} 2\,(x_i^T w - y_i)\; x_{i,j}$
- So:
  - $\nabla_w l(w) = 2\left(\begin{pmatrix} \sum_{i=1}^{n} x_i^T w x_{i,1} \\ \vdots \\ \sum_{i=1}^{n} x_i^T w x_{i,d} \end{pmatrix} - \begin{pmatrix} \sum_{i=1}^{n} y_i x_{i,1} \\ \vdots \\ \sum_{i=1}^{n} y_i x_{i,d} \end{pmatrix}\right)$

Linear Regression

- Goal: Minimize
  - $l(w) = (Xw - y)^T(Xw - y) = \sum_{i=1}^{n}\left(x_i^T w - y_i\right)^2$
- j'th entry of gradient:
  - $\frac{\partial}{\partial w_j} l(w) = \sum_{i=1}^{n} 2\,(x_i^T w - y_i)\; x_{i,j}$
- So:
  - $$\nabla_w l(w) = 2\left(\begin{pmatrix} \sum_{i=1}^{n} x_i^T w x_{i,1} \\ \vdots \\ \sum_{i=1}^{n} x_i^T w x_{i,d} \end{pmatrix} - \begin{pmatrix} \sum_{i=1}^{n} y_i x_{i,1} \\ \vdots \\ \sum_{i=1}^{n} y_i x_{i,d} \end{pmatrix}\right) =$$
    $$2\left(\begin{pmatrix} \sum_{i=1}^{n} x_i^T w x_{i,1} \\ \vdots \\ \sum_{i=1}^{n} x_i^T w x_{i,d} \end{pmatrix} - X^T y\right).$$

Linear Regression

- Goal: Minimize
  - $l(w) = (Xw - y)^T(Xw - y) = \sum_{i=1}^{n}\left(x_i^T w - y_i\right)^2$
- j'th entry of gradient:
  - $\frac{\partial}{\partial w_j} l(w) = \sum_{i=1}^{n} 2\,(x_i^T w - y_i)\; x_{i,j}$
- So:
  - $\nabla_w l(w) = 2\left(\begin{pmatrix} \sum_{i=1}^{n} x_i^T w x_{i,1} \\ \vdots \\ \sum_{i=1}^{n} x_i^T w x_{i,d} \end{pmatrix} - X^T y\right)$

Linear Regression

- Goal: Minimize
  - $l(w) = (Xw - y)^T(Xw - y) = \sum_{i=1}^{n}\left(x_i^T w - y_i\right)^2$
- j'th entry of gradient:
  - $\frac{\partial}{\partial w_j} l(w) = \sum_{i=1}^{n} 2\,(x_i^T w - y_i)\; x_{i,j}$
- So:
  - $\nabla_w l(w) = 2\left(\begin{pmatrix} \sum_{i=1}^{n} x_i^T w x_{i,1} \\ \vdots \\ \sum_{i=1}^{n} x_i^T w x_{i,d} \end{pmatrix} - X^T y\right) = 2\left(\begin{pmatrix} \sum_{i=1}^{n} (Xw)_i x_{i,1} \\ \vdots \\ \sum_{i=1}^{n} (Xw)_i x_{i,d} \end{pmatrix} - X^T y\right)$

Linear Regression

- Goal: Minimize
  - $l(w) = (Xw - y)^T(Xw - y) = \sum_{i=1}^{n}\left(x_i^T w - y_i\right)^2$
- j'th entry of gradient:
  - $\frac{\partial}{\partial w_j} l(w) = \sum_{i=1}^{n} 2\,(x_i^T w - y_i)\; x_{i,j}$
- So:
  - $\nabla_w l(w) = 2\left(\begin{pmatrix} \sum_{i=1}^{n}(Xw)_i x_{i,1} \\ \vdots \\ \sum_{i=1}^{n}(Xw)_i x_{i,d} \end{pmatrix} - X^T y\right)$

Linear Regression

- Goal: Minimize
  - $l(w) = (Xw - y)^T(Xw - y) = \sum_{i=1}^{n}\left(x_i^T w - y_i\right)^2$
- j'th entry of gradient:
  - $\frac{\partial}{\partial w_j} l(w) = \sum_{i=1}^{n} 2\,(x_i^T w - y_i)\; x_{i,j}$
- So:
  - $\nabla_w l(w) = 2\left(\begin{pmatrix} \sum_{i=1}^{n}(Xw)_i x_{i,1} \\ \vdots \\ \sum_{i=1}^{n}(Xw)_i x_{i,d} \end{pmatrix} - X^T y\right) = 2(X^T Xw - X^T y).$

Inner product j'th column of X with Xw

Linear Regression

- Goal: Minimize
  - $l(w) = (Xw - y)^T(Xw - y) = \sum_{i=1}^{n}\left(x_i^T w - y_i\right)^2$
- j'th entry of gradient:
  - $\frac{\partial}{\partial w_j} l(w) = \sum_{i=1}^{n} 2\,(x_i^T w - y_i)\; x_{i,j}$
- So:
  - $\nabla_w l(w) = 2(X^T X w - X^T y).$
- Set to 0:
  - $2(X^T X w - X^T y) = 0 \;\leftrightarrow$
  - $X^T X w = X^T y$
- If $X^T X$ is invertible:
  - $w = (X^T X)^{-1} X^T y.$

$(X^T X)^{-1} X^T$ called pseudo-inverse of X

Unique optimal solution!

Linear Regression

- Goal: Minimize
  - $l(w) = (Xw - y)^T(Xw - y) = \sum_{i=1}^{n}\left(x_i^T w - y_i\right)^2$
- If X^TX is invertible, then minimized for:
  - $w = (X^TX)^{-1}X^Ty$.

- Algorithm for least squares linear regression:
  - Compute and return $w = (X^TX)^{-1}X^Ty$

- Running time:
  - Compute X^TX: $O(d^2n)$
  - Compute inverse: $O(d^3)$
  - Compute X^Ty: $O(dn)$
  - Compute $((X^TX)^{-1})(X^Ty)$: $O(d^2)$

Total $O(d^2n+d^3)$

Linear Regression

- Goal: Minimize
  - $l(w) = (Xw - y)^T(Xw - y) = \sum_{i=1}^{n}\left(x_i^T w - y_i\right)^2$
- If X^TX is invertible, then minimized for:
  - $w = (X^TX)^{-1}X^Ty$.

- What if X^TX not invertible?
  - Simple exercise to deal with this case.
  - Idea: Delete columns (features) of X that are linearly dependent on others.

Linear Regression

- Goal: Minimize
  - $l(w) = (Xw - y)^T(Xw - y) = \sum_{i=1}^{n}\left(x_i^T w - y_i\right)^2$
- If X^TX is invertible, then minimized for:
  - $w = (X^TX)^{-1}X^Ty$.

- Computing gradient was tedious.
  - $\nabla_w l(w) = 2(X^TXw - X^Ty)$.

- Let us finish with more elegant proof.

Matrix of Derivatives

- Functions of multiple variables to multiple outputs:
  - If $l = (l_1, \ldots, l_k)$ is a function of variables $x_1, \ldots x_d$, i.e. from R^d to R^k.
  - Assume each l_i differentiable.

- Matrix of derivatives (a.k.a. Jacobian) is k x d:
  - $$\frac{\partial l}{\partial x} = \begin{pmatrix} \frac{\partial}{\partial x_1} l_1(x) & \cdots & \frac{\partial}{\partial x_d} l_1(x) \\ \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} l_k(x) & \cdots & \frac{\partial}{\partial x_d} l_k(x) \end{pmatrix}$$

- Gradient vector we looked at before, for functions from R^d to R, is the transpose of the matrix of derivatives.

Matrix of Derivatives

- Example:
  - If $l(x_1, x_2) = (x_1 + x_2^2, x_1^2, x_1 x_2)$ is from R^2 to R^3.

- Matrix of derivatives is 3 x 2:
  - $$\frac{\partial l}{\partial x} = \begin{pmatrix} \frac{\partial}{\partial x_1} l_1(x) & \cdots & \frac{\partial}{\partial x_d} l_1(x) \\ \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} l_k(x) & \cdots & \frac{\partial}{\partial x_d} l_k(x) \end{pmatrix} = \begin{pmatrix} 1 & 2x_2 \\ 2x_1 & 0 \\ x_2 & x_1 \end{pmatrix}$$

Matrix of Derivatives

- Matrix of derivatives is k x d: $l = (l_1, \ldots, l_k)$ is a function of variables $x_1, \ldots x_d$, i.e. from R^d to R^k.
  - $$\frac{\partial l}{\partial x} = \begin{pmatrix} \frac{\partial}{\partial x_1} l_1(x) & \cdots & \frac{\partial}{\partial x_d} l_1(x) \\ \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} l_k(x) & \cdots & \frac{\partial}{\partial x_d} l_k(x) \end{pmatrix}$$
- Convenient rules for matrix of derivates (Jacobian):

  Equivalent of $\frac{d}{dx} ax + b = a$

  - $l(x) = \mathrm{Ax} + \mathrm{b}$, $\frac{\partial l}{\partial x} = A$
  - A is k x d, x is d x 1, b is k x 1.
  - $l(x)$ is a function from R^d to R^k. Shapes match ☺

Matrix of Derivatives

- Matrix of derivatives is k x d: $l = (l_1, \dots, l_k)$ is a function of variables $x_1, \dots x_d$, i.e. from R^d to R^k.
  - $$\frac{\partial l}{\partial x} = \begin{pmatrix} \frac{\partial}{\partial x_1} l_1(x) & \cdots & \frac{\partial}{\partial x_d} l_1(x) \\ \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} l_k(x) & \cdots & \frac{\partial}{\partial x_d} l_k(x) \end{pmatrix}$$
- Convenient rules for matrix of derivates (Jacobian):

  Equivalent of $\frac{d}{dx} x^2 = 2x$

  - $l(x) = x^T x$, $\frac{\partial l}{\partial x} = 2x^T$
  - x is d x 1.
  - $l(x)$ is a function from R^d to R^1. Shapes match ☺

Matrix of Derivatives

- Matrix of derivatives is k x d: $l = (l_1, \ldots, l_k)$ is a function of variables $x_1, \ldots x_d$, i.e. from R^d to R^k.
  - $$\frac{\partial l}{\partial x} = \begin{pmatrix} \frac{\partial}{\partial x_1} l_1(x) & \cdots & \frac{\partial}{\partial x_d} l_1(x) \\ \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} l_k(x) & \cdots & \frac{\partial}{\partial x_d} l_k(x) \end{pmatrix}$$

- Convenient rules for matrix of derivates (Jacobian):
  - $\frac{\partial}{\partial x} \mathrm{Ax} + \mathrm{b} = \mathrm{A}$
  - $\frac{\partial}{\partial x} \mathrm{x}^{\mathrm{T}}\mathrm{x} = 2\mathrm{x}^{\mathrm{T}}$

Equivalent of $\frac{d}{dx} ax + b = a$

Equivalent of $\frac{d}{dx} x^2 = 2x$

Linear Regression

- Goal: Minimize
  - $l(w) = (Xw - y)^T(Xw - y) = \sum_{i=1}^{n}\left(x_i^T w - y_i\right)^2$
- Compute gradient as transpose of matrix of derivates.

- Matrix of derivatives:
  - $\frac{\partial}{\partial w}(Xw - y)^T(Xw - y) =$
- Using Chain rule:
  - $2(Xw - y)^T \frac{\partial}{\partial w}(\mathrm{X}\mathrm{w} - \mathrm{y}) =$
  - $2(Xw - y)^T X =$
  - $2(\mathrm{w}^{\mathrm{T}}\mathrm{X}^{\mathrm{T}}\mathrm{X} - \mathrm{y}^{\mathrm{T}}\mathrm{X})$.
- Gradient as transpose:
  - $2(\mathrm{w}^{\mathrm{T}}\mathrm{X}^{\mathrm{T}}\mathrm{X} - \mathrm{y}^{\mathrm{T}}\mathrm{X})^T = 2(X^T Xw - X^T y)$.

Rules:

- $\frac{\partial}{\partial x}\mathrm{A}\mathrm{x} + \mathrm{b} = \mathrm{A}$
- $\frac{\partial}{\partial x}\mathrm{x}^{\mathrm{T}}\mathrm{x} = 2\mathrm{x}^{\mathrm{T}}$

Same as we got earlier

Linear Regression

- Goal: Minimize
  - $l(w) = (Xw - y)^T(Xw - y) = \sum_{i=1}^{n}(x_i^T w - y_i)^2$
- $X = \begin{pmatrix} x_{1,1} & \cdots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{n,1} & \cdots & x_{n,d} \end{pmatrix}, y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}, w = \begin{pmatrix} w_1 \\ \vdots \\ w_d \end{pmatrix}$
- Analytic Solution:
  - Choose $w = (X^T X)^{-1} X^T y$.

